

Relatório Técnico

Universidade Federal do Amazonas (UFAM) - Instituto de Computação (ICOMP)

Disciplina: Algoritmos e Estruturas de Dados 2 (AED2)

Alunos: Alonso Ramos Brito Neto, Adrielly Silva de Souza, Gabriel Conceição dos Santos,
Pedro Vinicius Weil Montenegro II

1. Introdução e Metodologia

Este relatório detalha a implementação e a análise empírica de complexidade de diferentes algoritmos de busca e ordenação, conforme especificado no Trabalho 1 de AED2. O projeto foi desenvolvido em linguagem **C** para as estruturas de dados e algoritmos, utilizando o rigor de **Tipos Abstratos de Dados (TAD)** através da interface TAD-Vetorzao.h e sua implementação TAD-Vetorzao.c. Para a visualização dos resultados (tabelas e gráficos), foi construída uma ferramenta auxiliar em **Go** (visualizar_resultados.go) que processa os ficheiros JSON gerados pelo programa principal.

2. Questão 1: Busca Sequencial vs. Busca Binária

2.1. Detalhes de Implementação

O objetivo foi comparar a eficiência entre a Busca Sequencial e a Busca Binária num vetor de 1.000.000 (um milhão) de inteiros. Conforme a especificação:

- **Busca Sequencial:** Executada no vetor desordenado. A varredura é feita elemento a elemento a partir do índice 0 até encontrar a chave ou atingir o final (complexidade de tempo $O(n)$).
- **Busca Binária:** Executada no mesmo vetor, porém previamente ordenado. O algoritmo divide o espaço de busca pela metade a cada iteração, verificando o elemento central (complexidade de tempo $O(\log n)$).

2.2. Resultados e Comparações

Os resultados obtidos para as 30 execuções demonstram a enorme vantagem da Busca Binária para grandes volumes de dados ordenados:

Algoritmo	Tamanho (N)	Média (ms)	Desvio Padrão (ms)	Execuções
Busca Binária	1000000	0.006106	0.003927	30
Busca Sequencial	1000000	2.000540	1.108281	30

3. Questão 2: Busca Sequencial (Vetor vs. Lista Encadeada)

3.1. Detalhes de Implementação

Esta questão avaliou o impacto estrutural da alocação de memória. Um vetor contíguo foi comparado a uma Lista Encadeada alocada dinamicamente, ambos populados com os mesmos 1.000.000 inteiros desordenados da questão anterior. A Lista foi implementada no TAD usando a estrutura tipoNo (com dado e ponteiro prox). Foram realizadas as mesmas 30 buscas simultaneamente em ambas as estruturas utilizando exatamente as mesmas chaves.

3.2. Resultados e Comparações

Algoritmo	Tamanho (N)	Média (ms)	Desvio Padrão (ms)	Execuções
Busca em Vetor	1000000	1.934117	1.037717	30
Busca em Lista	1000000	3.242045	1.572590	30

Análise: Embora ambos os algoritmos pertençam à classe de complexidade teórica $O(n)$, a execução no Vetor é consideravelmente mais rápida que na Lista Encadeada. Isso ocorre devido à localidade de referência e uso da cache do processador. A Lista Encadeada possui nós espalhados na memória, causando atrasos frequentes durante a travessia dos ponteiros.

4. Questão 3: Ordenação de Vetores (100.000 elementos)

4.1. Detalhes de Implementação

Foi avaliada a performance de cinco algoritmos de ordenação num vetor com 100.000 inteiros aleatórios. O programa executou 10 rodadas, garantindo que os cinco algoritmos ordenassem exatamente a mesma massa de dados inicial a cada iteração.

4.2. Resultados e Comparações

Algoritmo	Tamanho (N)	Média (ms)	Desvio Padrão (ms)	Execuções
Quick Sort	100000	17.057575	6029.793469	10
Merge Sort	100000	19.853972	6761.773731	10
Insertion Sort	100000	6359.919761	4677.092407	10
Selection Sort	100000	11174.354258	9264.732751	10
Bubble Sort	100000	28154.383680	16224.534669	10

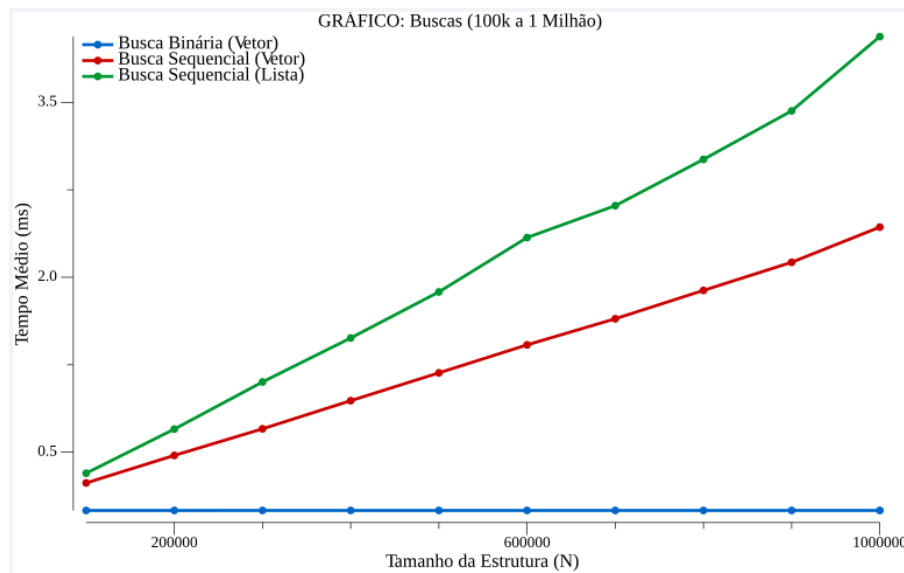
Análise: Quick e Merge Sort finalizaram rapidamente, demonstrando a eficiência da abordagem $O(n \log n)$. O Bubble Sort provou ser o mais ineficiente (desempenho quadrático

$O(n^2)$.

5. Comportamento Assintótico (Análise Gráfica)

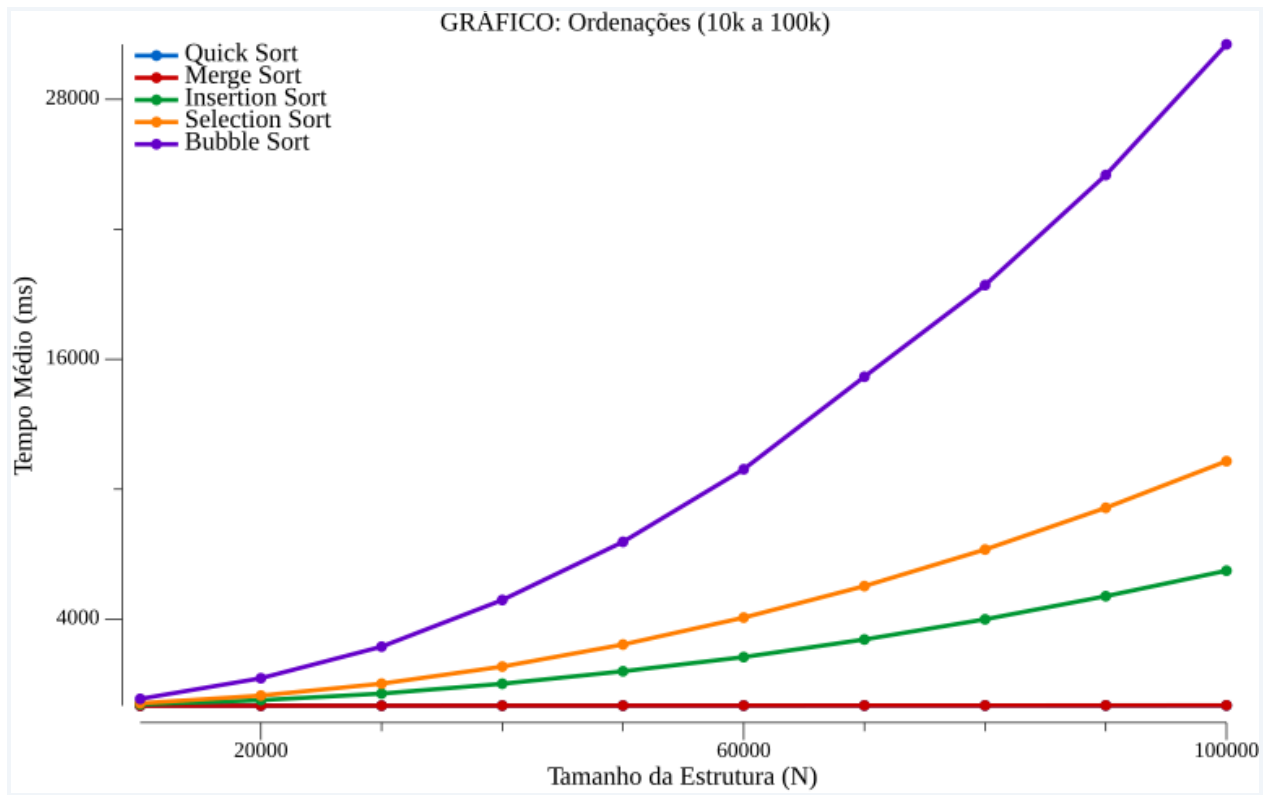
Os gráficos gerados ilustram visualmente a curva de crescimento do tempo de execução em função do tamanho da estrutura (N).

5.1. Gráfico de Buscas (100k a 1 Milhão)



O gráfico evidencia as três tendências de busca. A **Busca Binária** permanece constante perto do eixo horizontal, refletindo a sua complexidade logarítmica. A **Busca Sequencial no Vetor** cresce linearmente. Já a **Busca Sequencial na Lista** exibe também um crescimento linear, mas com uma inclinação bastante superior devido ao sobrecusto de acesso à memória.

5.2. Gráfico de Ordenações (10k a 100k)



Este gráfico comprova as diferenças teóricas entre os algoritmos. O **Bubble Sort**, o **Selection Sort** e o **Insertion Sort** desenharam parábolas acentuadas, o que caracteriza claramente o tempo quadrático. Em contrapartida, as curvas do **Quick Sort** e do **Merge Sort** são tão achatadas e próximas do eixo inferior que aparentam um comportamento constante nesta escala, destacando a extrema superioridade dos algoritmos baseados em divisão e conquista.

6. Conclusão

Os dados recolhidos confirmam empiricamente as teorias de complexidade assintótica (Notação Big-O). A distinção entre estruturas de dados de acesso aleatório (vetores) e sequencial (listas) mostrou-se vital no tempo de resposta, assim como a escolha correta do algoritmo de ordenação para grandes massas de dados.